УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 9

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ, СОДЕРЖАЩИХ ПОЛЬЗОВАТЕЛЬСКИЕ ФУНКЦИИ

ЛАБОРАТОРНАЯ РАБОТА № 9.2

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ, СОДЕРЖАЩИХ РЕКУРСИВНЫЕ И ЛЯМБДА-ФУНКЦИИ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейших программ, содержащих рекурсивные и лямбда-функции

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Анонимные функции

Помимо обычных, язык Python позволяет использовать анонимные функции, которые также называются **лямбда-функциями**. Анонимная функция описывается с помощью ключевого слова **lambda** по следующей схеме:

lambda [<Параметр1>[, ..., <ПараметрN>]]: <Возвращаемое значение>

После ключевого слова **lambda** можно указать передаваемые параметры. В качестве параметра **<Возвращаемое значение>** указывается выражение, результат выполнения которого будет возвращен функцией. Как видно из схемы, у таких функций нет имени; по этой причине их и называют анонимными.

В качестве значения анонимная функция возвращает ссылку на объект-функцию, которую можно сохранить в переменной или передать в качестве параметра в другую функцию. Вызвать анонимную функцию можно, как и обычную, с помощью круглых скобок, внутри которых расположены передаваемые параметры. Пример использования анонимных функций приведен ниже:

```
f1 = lambda: 10+20 # Функция без параметров
f2 = lambda x,y: x + y # Функция с двумя  параметрами
f3 = lambda x,y,z: x + y + z # Функция  с тремя параметрами
print(f1()) # Выведет: 30
print(f2(5, 10)) # Выведет: 15
print(f3(5, 10, 30)) # Выведет: 45
```

Как и в обычных функциях, некоторые параметры анонимных функций могут быть необязательными. Для этого параметрам в определении функции присваивается значение по умолчанию:

```
f = lambda x,y=2: x + y
```

```
print(f(5))      # Выведет: 7
print(f(5, 6))   # Выведет: 11
```

Чаще всего анонимную функцию не сохраняют в переменной, а сразу передают в качестве параметра в другую функцию. Например, метод списков **sort()** позволяет указать пользовательскую функцию в параметре **key**. Отсортируем список без учета регистра символов, указав в качестве параметра функцию:

```
arr = ["единица1", "Единый", "Единица2"]
arr.sort (key=lambda s: s.lower())
for i in arr:
    print(i, end=" ")
# Результат выполнения: единица1 Единица2 Единый
```

4.2 Функции-генераторы

Функцией-генератором называется функция, которая может возвращать одно значение из нескольких значений на каждой итерации. Приостановить выполнение функции и превратить функцию в генератор позволяет ключевое слово **yield**. В качестве примера напишем функцию, которая возводит элементы последовательности в указанную степень:

```
def func(x, y):
    for i in range(1, x + 1):
        yield i ** y
for n in func(10, 2): print(n, end=" ")
# Выведет: 1 4 9 16 25 36 49 64 81 100
print() # Вставляем пустую строку
for n in func(10, 3): print(n, end=" ")
# Выведет: 1 8 27 64 125 216 343 512 729 1000
```

Функции-генераторы поддерживают метод **__next__()**, который позволяет получить следующее значение. Когда значения заканчиваются, метод возбуждает исключение **StopIteration**. Вызов метода **__next__()** в цикле **for** производится незаметно для нас.

В качестве примера перепишем предыдущую программу и используем метод **__next__()** вместо цикла **for**:

```
def func(x, y):
    for i in range(1, x + 1):
        yield i ** y
i = func(3, 3)
print(i.__next__()) # Выведет: 1 (1 ** 3)
print(i.__next__()) # Выведет: 8 (2 ** 3)
print(i.__next__()) # Выведет: 27 (3 ** 3)
print(i.__next__()) # Исключение StopIteration
```

Получается, что с помощью обычных функций мы можем вернуть все значения сразу в виде списка, а с помощью функций-генераторов - только одно значение за раз. Эта особенность очень полезна при обработке большого

количества значений, т. к. не понадобится загружать весь список со значениями в память.

В **Python 3.3** появилась возможность вызвать одну функцию-генератор из другой. Для этого применяется расширенный синтаксис ключевого слова **yield**:

yield from <*Вызываемая функция-генератор*>

Рассмотрим следующий пример. Пусть у нас есть список чисел, и нам требуется получить другой список, включающий числа в диапазоне от 1 до каждого из чисел в первом списке. Чтобы создать такой список, мы напишем код. показанный ниже:

```
def gen(l):
    for e in l:
        yield from range(1, e + 1)
l = [5, 10]
for i in gen([5, 10]): print(i, end = " ")
```

Здесь мы в функции-генераторе **gen** перебираем переданный ей в качестве параметра список и для каждого его элемента вызываем другую функцию-генератор. В качестве последней выступает выражение, создающее диапазон от 1 до значения этого элемента, увеличенного на единицу (чтобы это значение вошло в диапазон). В результате на выходе мы получим вполне ожидаемый результат:

1 2 3 4 5 1 2 3 4 5 6 7 8 9 10

Усложним задачу, включив в результирующий список числа, умноженные на 2. Код, выполняющий эту задачу, показан ниже:

```
def gen2(n):
    for e in range(1, n + 1):
        yield e * 2
def gen(l):
    for e in l:
        yield from gen2(e)

l = [5, 10]
for i in gen([5, 10]): print(i, end = " ")
```

Здесь мы вызываем из функции-генератора **gen** написанную нами самими функцию-генератор **gen2**. Она создает диапазон, перебирает все входящие в него числа и возвращает их умноженными на 2. Результат работы приведенного в листинге кода таков:

2 4 6 8 10 2 4 6 8 10 12 14 16 18 20

4.3 Декораторы функций

Декораторы позволяют изменить поведение обычных функций - например, выполнить какие-либо действия перед выполнением функции. Рассмотрим это на примере:

```
def deco(f): # Функция-декоратор
    print("Вызвана функция  func()")
    return f #  Возвращаем ссылку на функцию
```

```
@deco
def func (x):
    return  "x = {0}".format(x)
print (func(10))
```

В этом примере перед определением функции **func()** указывается название функции **deco()** с предваряющим символом **@**:

```
@deco
```

Таким образом, функция **deco()** становится декоратором функции **func()**. В качестве параметра функция-декоратор принимает ссылку на функцию, поведение которой необходимо изменить, и должна возвращать ссылку на ту же функцию или какую-либо другую. Предыдущий пример эквивалентен следующему коду:

```
def deco(f):
    print("Вызвана функция func()")
    return f
def func(x):
    return "x = {0}".format(x)
# Вызываем функцию func() через функцию deco()
print (deco(func) (10))
```

Перед определением функции можно указать сразу несколько функций-декораторов. Для примера обернем функцию **func()** в два декоратора: **deco1()** и **deco2()**:

```
def deco1(f):
    print("Вызвана функция deco1()")
    return f
def deco2(f):
    print("Вызвана функция deco2()")
    return f
@deco1
@deco2
def func (x):
    return "x = {0}".format(x)
print(func(10))
```

Использование двух декораторов эквивалентно следующему коду:

```
func = deco1(deco2(func))
```

Здесь сначала будет вызвана функция **deco2()**, а затем функция **deco1()**. Результат выполнения будет присвоен идентификатору **func**.

В качестве ещё одного примера использования декораторов рассмотрим выполнение функции только при правильно введенном пароле:

```
passw = input("Введите пароль: ")
def test_passw(p):
    def deco(f):
        if p == "10": # Сравниваем пароли
            return f
        else:
            return lambda: "Доступ закрыт"
```

```
    return deco # Возвращаем функцию-декоратор
@test_passw(passw)
def func():
    return  "Доступ открыт"
print(func()) # Вызываем функцию
```

В этом примере после символа **@** указана не ссылка на функцию, а выражение, которое возвращает декоратор. Иными словами, декоратором является не функция **test_passw()**, а результат ее выполнения (функция **deco()**). Если введенный пароль является правильным, то выполнится функция **func()**, в противном случае будет выведена надпись **"Доступ закрыт"**, которую возвращает анонимная функция.

4.4 Рекурсия. Вычисление факториала

Рекурсия – это возможность функции вызывать саму себя. Рекурсию удобно использовать для перебора объекта, имеющего заранее неизвестную структуру, или для выполнения неопределенного количества операций. В качестве примера рассмотрим вычисление факториала.

```
def factorial(n):
    if n == 0 or n == 1: return 1
    else:
        return n * factorial(n - 1)
while True:
    x = input("Введите число: ")
    if x.isdigit(): # Если строка содержит только цифры
        x = int(x) # Преобразуем строку в число.
        break # Выходим из цикла
    else:
        print ("Вы ввели не число!")
print("Факториал числа {0} = {1}".format(x,factorial(x)))
```

Впрочем, проще всего для вычисления факториала воспользоваться функцией factorial() из модуля math.

Пример:

```
import math
print( math.factorial(5), math.factorial(6))
(120, 720)
```

4.5 Вложенные функции

Одну функцию можно вложить в другую функцию, причем уровень вложенности не ограничен. При этом вложенная функция получает свою собственную локальную область видимости и имеет доступ к переменным, объявленным внутри функции, в которую она вложена (функции-родителя). Рассмотрим вложение функций на примере.

```
def func1(x):
```

```
    def func2():
        print(x)
    return func2
f1 = func1(10)
f2 = func1(99)
f1() # Выведет: 10
f2() # Выведет: 99
```

Здесь мы определили функцию **func1()**, принимающую один параметр. Внутри функции **func1()** определена вложенная функция **func2()**. Результатом выполнения функции **func1()** будет ссылка на эту вложенную функцию. Внутри функции **func2()** мы производим вывод значения переменной **x**, которая является локальной в функции **func1()**. Таким образом, помимо локальной, глобальной и встроенной областей видимости, добавляется ***вложенная область видимости***. При этом поиск идентификаторов вначале производится внутри вложенной функции, затем внутри функции-родителя, далее в функциях более высокого уровня и лишь потом в глобальной и встроенных областях видимости. В нашем примере переменная **x** будет найдена в области видимости функции **func1()**.

Следует учитывать, что сохраняются лишь ссылки на переменные, а не их значения в момент определения функции. Например, если после определения функции **func2()** произвести изменение переменной **x**, то будет использоваться это новое значение:

```
def func1(x):
    def func2():
        print(x)
    x = 30
    return func2
f1 = func1(10)
f2 = func1(99)
f1() # Выведет: 30
f2() # Выведет: 30
```

Обратите внимание на результат выполнения. В обоих случаях мы получили значение 30. Если необходимо сохранить именно значение переменной при определении вложенной функции, следует передать значение как значение по умолчанию:

```
def func1(x):
    def func2(x=x): # #  Сохраняем текущее значение, а не ссылку
        print(x)
    x = 30
    return func2
f1 = func1(10)
f2 = func1(99)
f1() # Выведет: 10
f2() # Выведет: 99
```

Теперь попробуем из вложенной функции **func2()** изменить значение переменной **x**, объявленной внутри функции **func1()**. Если внутри

функции **func2()** присвоить значение переменной **x**, то будет создана новая локальная переменная с таким же именем. Если внутри функции **func2()** объявить переменную как глобальную и присвоить ей значение, то изменится значение глобальной переменной, а не значение переменной **x** внутри функции **func1()**. Таким образом, ни один из изученных ранее способов не позволяет из вложенной функции изменить значение переменной, объявленной внутри функции-родителя. Чтобы решить эту проблему, следует объявить необходимые переменные с помощью ключевого слова **nonlocal**.

```
def func1(a):
    x = a
    def func2(b):
        nonlocal x # Объявляем переменную как nonlocal
        print(x)
        x = b # Можем изменить значение x в func1()
    return func2
f = func1(10)
f(5)  # Выведет: 10
f(12) # Выведет: 5
f(3)  # Выведет: 12
```

При использовании ключевого слова **nonlocal** следует помнить, что переменная обязательно должна существовать внутри функции-родителя. В противном случае будет выведено сообщение об ошибке.

4.6 Аннотации функций

В **Python 3** функции могут содержать аннотации, которые вводят новый способ документирования. Теперь в заголовке функции допускается указывать предназначение каждого параметра, данные какого типа он может принимать, а также тип возвращаемого функцией значения. Аннотации имеют следующий формат:

```
def <Имя функции>(
  [<Параметр1> [: <Выражение>] [= <Значение по умолчанию>][, ...,
   <ПараметрN> [: <Выражение>] [= <Значение по умолчанию>] ] ]
  ) -> <Возвращаемое значение>:
    <Тело функции>
```

В параметрах ***<Выражение>*** и ***<Возвращаемое значение>*** можно указать любое допустимое выражение языка **Python**. Это выражение будет выполнено при создании функции. Пример указания аннотаций:

```
def func(a: "Параметр1", b: 10  +  5 =  3) -> None:
  print(a, b)
```

В этом примере для переменной **a** создано описание ***"Параметр1"***. Для переменной **b** выражение 10 + 5 является описанием, а число 3 - значением параметра по умолчанию. Кроме того, после закрывающей круглой скобки указан тип возвращаемого функцией значения: **None**. После создания функции все выражения будут выполнены, и результаты сохранятся в виде словаря в

атрибуте **__annotations__** объекта функции. Для примера выведем значение этого атрибута:

```
def func(a: "Пример1", b: 10 + 5 = 3) -> None: pass
print(func.__annotations__)
{'a': 'Пример1', 'b': 15, 'return': None}
```

4.7 Пример решения задачи на языке программирования Python

Задача. Напишите рекурсивную функцию, которая определяет, сколько раз заданное число встречается в списке.

Решение задачи:

```
def count_occurrences(lst, x, i=0):
    # базовый случай: дошли до конца списка
    if i == len(lst):
        return 0
    # если текущий элемент равен x
    if lst[i] == x:
        return 1 + count_occurrences(lst, x, i + 1)
    else:
        return count_occurrences(lst, x, i + 1)
# пример использования
a = [1, 2, 3, 2, 4, 2, 5]
print(count_occurrences(a, 2))  # 3
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Напишите рекурсивную функцию factorial(n), вычисляющую факториал числа.

2 Напишите рекурсивную функцию sum_digits(n), вычисляющую сумму цифр числа.

3 Напишите рекурсивную функцию fibonacci(n), возвращающую n-й элемент последовательности Фибоначчи.

4 Напишите рекурсивную функцию reverse_string(s), переворачивающую строку.

5 Напишите рекурсивную функцию count_down(n), которая выводит числа от n до 1.

6 Напишите рекурсивную функцию power_recursive(a, n), вычисляющую a^n.

7 Напишите рекурсивную функцию gcd(a, b), вычисляющую наибольший общий делитель (алгоритм Евклида).

8 Напишите рекурсивную функцию list_sum(lst), возвращающую сумму элементов списка.

9 Напишите рекурсивную функцию print_nested_list(lst), выводящую все элементы вложенного списка.

10 Напишите рекурсивную функцию decimal_to_binary(n), преобразующую число в двоичную строку.

11 Напишите рекурсивную функцию sum_even_numbers(lst), которая вычисляет сумму всех чётных чисел в списке (включая элементы вложенных списков).

12 Напишите рекурсивную функцию is_palindrome(s), которая проверяет, является ли строка палиндромом (читается одинаково слева направо и справа налево).

13 Напишите рекурсивную функцию find_max(lst), которая находит максимальный элемент в списке (в том числе во вложенных списках).

14 Напишите рекурсивную функцию flatten_list(lst), которая преобразует вложенный список в одномерный список.

15 Напишите рекурсивную функцию count_elements(lst), которая определяет общее количество элементов в списке, включая элементы вложенных списков.

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Используя lambda, создайте функцию для вычисления квадрата числа.

2 Используя lambda, создайте функцию, которая проверяет, является ли число чётным.

3 Используя map() и lambda, преобразуйте все элементы списка в их квадраты.

4 Используя filter() и lambda, выберите из списка только положительные числа.

5 Используя lambda, отсортируйте список кортежей по второму элементу.

6 Используя reduce() и lambda, вычислите сумму всех элементов списка.

7 Используя lambda, создайте функцию, которая возвращает произведение двух чисел.

8 Используя lambda, отсортируйте список строк по их длине.

9 Используя lambda, создайте выражение, которое возвращает True, если число делится на 3 и 5 одновременно.

10 Используя map() и lambda, преобразуйте список строк в список их длин.

11 Используя lambda и filter(), выберите из списка слов только те, которые начинаются с заданной буквы.

12 Используя lambda и map(), преобразуйте список температур из градусов Цельсия в градусы Фаренгейта по формуле F = C * 9/5 + 32.

13 Используя lambda, создайте выражение, которое возвращает строку в верхнем регистре, если её длина больше 5, иначе — в нижнем.

14 Используя lambda, создайте функцию, которая возвращает максимальное из двух чисел.

15 Используя lambda, создайте функцию, которая возвращает длину строки.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы
6.2 Цель лабораторной работы
6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Дайте определение понятию «Декораторы функций» в языке программирования Python.
7.2 Дайте определение понятию «Рекурсия» в языке программирования Python.
7.3 Опишите использование лямбда-функций в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.
2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.
3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.